



Department of Computer Science and Engineering,
PES University, Bangalore, India

UE23CS341B: Compiler Design

Prakash C O, Associate Professor, Department of CSE

Types of compilers

The main types of computer compilers are:

1. One pass compiler
2. Multi pass compiler
3. Decompiler
4. Source-to-source compiler
5. Cross compiler
6. JIT Compiler
7. Native Compiler
8. Compiler-compiler

One-Pass Compiler

- Processes the source code only once, generating target code directly during that single traversal.
- **Advantages:**
 - Faster compilation (only one pass).
 - Simpler design and implementation.
 - Useful for small or resource-constrained systems.
- **Disadvantages:**
 - Limited optimization opportunities.
 - Harder to handle complex language features (forward references, type checking across scopes).
- **Example languages:** Pascal can be implemented with a one-pass compiler.

Multi-Pass Compiler

- Processes the source code or intermediate representation multiple times, each pass handling a specific task (e.g., syntax analysis, semantic analysis, optimization, code generation).
- **Advantages:**
 - Enables sophisticated optimizations for high-quality machine code.

- Easier to support complex language features (like Java, C++, or modern functional languages).
 - Clear separation of concerns between passes.
- **Disadvantages:**
 - Slower compilation (multiple traversals).
 - More complex to design and maintain.
- **Example languages:** Java typically requires a multi-pass compiler.

Decompiler (Opposite of a compiler)

- A decompiler is a program that takes **low-level code** (machine code, bytecode, or assembly) and translates it back into a **higher-level language** (like C, Java, or pseudo-code).
- Why Decompilers are important
 - **Reverse engineering:** Used to understand how software works when source code is unavailable.
 - **Software maintenance:** Useful when source code is lost or corrupted.
 - **Security research:** Analysts use decompilers to inspect malware and understand its behavior.

Source-to-Source Compiler

- A **Source-to-Source Compiler** (also called a **transpiler**) is a type of compiler that takes source code written in one high-level programming language and translates it into source code of another high-level language, rather than directly into machine code or bytecode.
- Often used for language migration or cross-platform development.

Cross compiler

- A cross compiler is a compiler that runs on one machine (the *host*) but generates executable code for another machine (the *target*).
- The compiler itself doesn't produce code for the system it's running on, but for a different architecture and operating system.

How It Works

- **Host machine (A):** Where the compiler runs.
- **Target machine (B):** Where the generated code will execute.
- **Example:** A compiler running on a Windows 10 PC that produces binaries for an Android smartphone (ARM architecture).

Why Cross Compilers Are Needed

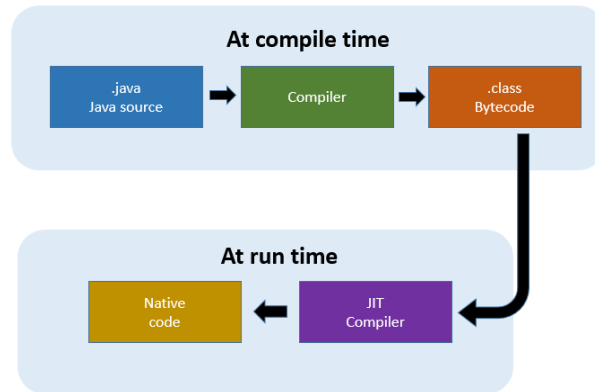
- **Embedded systems:** Microcontrollers and IoT devices often lack the resources to host a compiler. Developers use cross compilers on a powerful PC to generate code for them.
- **New architectures:** When porting software to new hardware, cross compilers allow developers to build code before native compilers exist.

Examples

- **GCC:** Can be configured to cross compile for many platforms (e.g., x86 → ARM).
- **Clang/LLVM:** Supports cross compilation with proper target specifications.
- **Android NDK toolchain:** Provides cross compilers to build apps on desktop machines for Android devices.

JIT Compiler

- A JIT compiler is a component of the Java Virtual Machine (JVM) that improves performance by compiling Java bytecode into native machine code *at runtime*.
- **Benefits**
 - **Performance boost:** Native machine code runs much faster than interpreted bytecode.
 - **Portability retained:** Developers still write once in Java, and the JVM ensures it runs anywhere, with JIT improving speed locally.



Native Compiler

- A native compiler is a compiler that converts high-level programming languages into machine code for the operating system or system architecture on which the compiler is running.
- **Benefits:** Native compilers can provide benefits such as faster code loading, better code security, and preventing reverse engineering.
- Cross compilers produce code for a different platform than the one on which the compiler is running. For example, a cross compiler can run on a PC but generate code for Android devices.

Native vs Cross Compiler

Type	What it does	Example
Native Compiler	Compiles code for the same system it runs on	GCC on Linux producing Linux binaries
Cross Compiler	Compiles code for a <i>different</i> system/architecture	ARM cross-compiler on x86 PC producing ARM binaries

Self-hosting Compiler (A compiler is *self-hosting* if it can compile itself.)

- A **self-hosting compiler** is a compiler that is written in the same programming language that it compiles. In other words, the compiler is capable of compiling its own source code.
- **Example:** A C compiler written in C, or a Rust compiler written in Rust.

In some of these cases, the initial implementation was not self-hosted, but rather, written in another language (or even in machine language); in other cases, the initial implementation was developed using bootstrapping.

Bootstrapping (building a system using the system itself.)

- **Bootstrapping** is the process of developing a self-hosting compiler.
- How Bootstrapping Works

- Stage 1 – **Initial compiler**
 - A simple compiler (or interpreter) is written in another language (often assembly or C).
- Stage 2 – **Rewrite in the new language**
 - Developers rewrite the compiler in the new language itself.
 - The initial compiler is used to compile this new compiler.
- Stage 3 – **Self-hosting**
 - Once the compiler can compile its own source code, it becomes self-hosting.
- **Examples:**
 - **C:** Early C compilers were bootstrapped using assembly, then rewritten in C.
 - **Go:** The Go compiler was initially written in C, later rewritten in Go itself.
 - Assemblers were the first language tools to bootstrap themselves.

Compiler-compiler

- A **compiler-compiler** or **compiler generator** is a programming tool that creates a parser, interpreter, or compiler from some form of formal description of a programming language and machine.
- The most common type of compiler-compiler is called a parser generator. It handles only syntactic analysis.
- **Examples:**
 - **YACC** (Yet Another Compiler Compiler) is a classic parser generator tool developed in the 1970s for Unix systems. **Generates parsers only in C.**
 - **Bison** is a parser generator, very **similar to YACC**, that is part of the GNU Project. **Generates parsers in C, C++, and Java.**
 - **ANTLR** (ANother Tool for Language Recognition). **Written in Java**, but generates parsers in multiple languages (Java, C#, Python, JavaScript, Go, Swift, etc.).

Syntactic Sugar

- It refers to **language features or syntax that make code easier to read or write**, but doesn't add new functionality.
- It's "sweet" because it makes programming more pleasant without changing the underlying semantics.

Why is it used?

- Improves **readability** and **developer productivity**.
- Makes code closer to natural language or intuitive patterns.

Examples:

- **Python:**
 - `for x in list:` is syntactic sugar for using an iterator explicitly.
 - List comprehensions `[x for x in iterable]` instead of writing loops.
- **Java:**
 - Enhanced for-loop: `for (int x : arr)` is sugar for using an iterator.
- **JavaScript:**
 - Arrow functions `() => {}` instead of `function() {}`.

Desugaring:

- The compiler or interpreter **translates sugared syntax into more fundamental constructs** before execution.
- Example:
 - $a[i] \rightarrow *(a + i)$ in C
 - `for x in list:  $\rightarrow$  iter_obj = iter(list); while True: ...` in Python

Examples:

- In C language: $a[i]$ is a syntactic sugar for $*(a+i)$
 - In C language: $a+=b$ is equivalent to $a = a + b$
 - In C# language: `var x = expr` (compiler deduces type of x from expr)
 - In Perl language: `unless(condition) {...}` is a syntactic sugar for `if (not condition) {...}`
-

Types of Compilers (in brief)

1. One-Pass Compiler

- Processes source code in **one pass**.
- **Advantages:** Faster compilation, simpler design.
- **Disadvantages:** Limited optimization.
- Example: Pascal can use one-pass compilation.

2. Two-Pass Compiler

A **two-pass compiler** is a type of **multi-pass compiler** that processes the source code in **two distinct passes** over the input program. Each pass performs a specific set of tasks:

• Pass 1: Analysis Phase (Front end)

- Reads the source code and performs:
 - **Lexical Analysis** (tokenization)
 - **Syntax Analysis** (parsing)
 - **Semantic Analysis**
- Builds **intermediate representation (IR)** and symbol table.
- Checks for errors like undeclared variables, type mismatches ...

Pass 2: Synthesis Phase (Back end)

- Uses the intermediate representation from Pass 1.
- Performs:
 - **Code Optimization**
 - **Code Generation** (machine code or bytecode)
- Outputs the final target code.

3. Multi-Pass Compiler

- Processes source code in **multiple passes**.
- **Advantages:** Better optimization, supports complex languages.
- **Disadvantages:** Slower than one-pass.
- Example: Java requires multi-pass compilation.

3. Decompiler

- Converts **low-level code** → **high-level code**.
- Used in **reverse engineering**.

4. Source-to-Source Compiler (Transpiler)

- Converts **one high-level language** → **another high-level language**.
- Example: TypeScript → JavaScript.

5. Cross Compiler

- Runs on **Machine A**, generates code for **Machine B**.
- Used for **embedded systems, multi-platform development**.
- Example: GCC can be configured for cross-compilation.

6. JIT Compiler (Just-In-Time)

- Converts **bytecode** → **machine code at runtime**.
- Improves performance in languages like **Java**.

7. Native Compiler

- Converts high-level code into **machine code for the same platform**.
- **Benefits**: Faster execution, better security.

8. Compiler-Compiler

- A tool that **generates compilers or parsers** from formal language descriptions.
- **Examples**: YACC, Bison and ANTLR

Related Concepts

- **Self-Hosting Compiler**: A compiler that can compile its own source code.
- **Bootstrapping**: Technique to build a self-compiling compiler starting from a minimal version.
- **Cross Compilation Issues**:
 - Toolchain complexity, headers/libraries management, special options like --sysroot.

Optimizing Compiler

- An optimizing compiler is a compiler that attempts to **minimize or maximize certain attributes** of an executable program.
- **Common Goals**:
 - Minimize execution time (most common).
 - Minimize memory usage.
 - Minimize power consumption (important for portable devices).
- **How It Works**:
 - Uses **optimizing transformations**: algorithms that take a program and transform it into a **semantically equivalent program** that uses fewer resources.

- These transformations are applied in **multiple stages** during compilation.
-

Note:

Cross compilation issues

- In GCC world, every host/target combination has its own set of binaries, headers, libraries, etc. So, it's usually simple to download a package with all files in, unzip to a directory and point the build system to that compiler, that will know about its location and find all it needs to when compiling your code.
 - On the other hand, **Clang/LLVM is natively a cross-compiler**, meaning that one set of programs can compile to all targets by setting the -target option. That makes it a lot easier for programmers wishing to compile to different platforms and architectures, and for compiler developers that only have to maintain one build system, and for OS distributions, that need only one set of main packages.
 - But, as is true to any cross-compiler, and given the complexity of different architectures, OS's and options, it's not always easy finding the headers, libraries or binutils to generate target specific code. So you'll need special options to help Clang understand what target you're compiling to, where your tools are, etc.
 - Another problem is that compilers come with standard libraries only (like compiler-rt, libcxx, libgcc, libm, etc), so you'll have to find and make available to the build system, every other library required to build your software, that is specific to your target. It's not enough to have your host's libraries installed.
 - Finally, not all toolchains are the same, and consequently, not every Clang option will work magically. Some options, like --sysroot (which effectively changes the logical root for headers and libraries), assume all your binaries and libraries are in the same directory, which may not be true when your cross-compiler was installed by the distribution's package management. So, for each specific case, you may use more than one option, and in most cases, you'll end up setting include paths (-I) and library paths (-L) manually.
-